

1. INTRODUCTION

- **Software** are computer programs, procedures, and associated documentation and data pertaining to the operation of a computer system.
- **Software engineering** is the application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of the software.
- **Generic softwares (product)** - developed for a broad market (ex. ms word).
- **Bespoke softwares (custom)** - build for a single customer.
- **System software** - operate and controls computer hardware (**System management programs** - OS **System support programs** - Utilities **System development programs** - language translators)
- **Application software** - performs a specific task for end users (**General purpose programs** - ms word **Application specific programs** - Accounting software)
- **Business software** - management information system software that access one or more large database containing business information.
- **Embedded software** - resides in read-only memory and is used to control product and system for the customer and industrial markets.
- **Web-based software** - The network becomes a massive computer providing an almost unlimited software resources that can be accessed by anyone with a modem.
- **AI Software** - makes use of non-numerical algorithms to solve the complex problems that are not amenable to computing or straightforward analysis.
- **Scientific or Engineering software** - Used for modeling, simulation, or analysis in fields like physics, chemistry, and engineering.
- **PC Software** - Word processing, spreadsheets, computer graphic, multimedia management.
- **Software are hard to build since:** Difficult to specify - customers often don't know what they want until they see it. Requirements changes constantly. Is intangible. Exhaustive testing is impossible. Small changes have large effects. Software development is young; measurable techniques not well calibrated. Replication of software is trivial, software becomes elastic and more complex over time.
- **Why software fails:** Increasing system complexity, lack of software engineering discipline, unclear/constantly changing requirements, poor communication among stakeholders, unrealistic timelines/budgets, lack of testing discipline, scope creep (features keep expanding), no formal engineering process.
- **Software cost:** Software costs more than hardware. 20-40% of budget goes to development while 60-80% goes to maintenance. A major goal of software engineering is to make software that's reliable, maintainable, and scalable without ballooning the cost.
- **Degradation:** Software has no physical degradation, but still may fail due to small changes. Software logically degrade (Spike pattern - spikes at every update). Hardware degrades physically (Bath-tub curve - high at installation and end of life).
- **Software crisis:** Over budget, late or never delivered, wrong product, unmaintainable, no discipline. **Causes:** Increasing software complexity, Lack of SE discipline, Immaturity of software as a field, Inherent nature of software. **Solutions:** Clear requirements, prototyping, engineering process, quality focus, tool support.
- **Product quality:** (characteristics of the final product) usability, reliability, efficiency, maintainability, security.
- **Process quality:** (characteristics of how it was built) standards followed consistently, code reviews and inspections carried out, testing at every stage of development, documentation kept updated, version control used throughout.
- **Professional & Ethical responsibility:** confidentiality(respect sensitive user and organizational data), competence(do not accept work you are not qualified to do), intellectual property(respect copy rights, licenses), computer misuse(do not abuse system access, install malware), public safety(safety critical software demands the highest rigour-lives may depend on it), honesty (be transparent about progress, problems and limitations).
- **ACM/IEEE Code of Ethics:** Public - welfare/safety first; Client & Employer - serve con-

sistent with public interest; Product - highest professional standards; Judgement - integrity & independence; Management - promote ethical management; Profession - advance integrity/reputation of SE; Colleagues - fair, honest, supportive; Self - lifelong learning & ethical practice.

- **Core principles of SE:** use a defined process, ensure dependability, start with clear requirements, reuse when possible, incremental and agile development, service oriented architecture, rich interfaces.
- **SDLC:** Requirements, design, implementation, testing, deployment, maintenance.
- **Stakeholders:** end users, domain experts, regulators, managers, technical staff, operations and support, designers.
- **Coping with change:** *change anticipation:* build activities into process that predict changed before significant rework is required. used in spiral and prototyping. *change tolerance:* design the process and software so changes can be easily incorporated. used in agile and incremental. *continuous refactoring:* continuously improve internal code structure without changing external behaviors. used in agile, XP, DevOps
- **Software process:** a structured set of activities required to develop a software system. Activities(the work that is carried out), Products(what each activity produces), Roles(who carries out the work), Pre Post conditions (what must be true before and activity starts)

Why defined process: common understanding(to make sure everyone understand activities), consistency and quality(reduces defects and makes quality repeatable), measurable progress(it provides milestones, deliverables and checkpoints that make progress visible), adaptability(help understand that different system need different approaches), professional practice(this is what defines the discipline of SE).

2. PLAN-DRIVEN CULTURE

- Plan everything before coding; requirements, design, schedule fixed; progress measured with plan; change is controlled & expensive; emphasis on formal documentation & reviews; best when requirements are fixed.
- **Use plan-driven** when requirements are clear and stable; building safety critical system; large, distributed, contractual teams; customer availability at milestone events; long term fixed contract timelines; well understood tech stacks; low risk levels.
- **WATERFALL** - Requirement analysis & definition; system & software design; implementation & unit testing; integration & system testing; operation & maintenance.
- **Strengths** are simple to understand, heavy documentation makes easy to hand over;
- **Use when** requirements are fixed or unlikely to change, short projects, safety-critical systems, replacing a well known existing system;
- **Limitations** occur due to requirements never stay still, real projects don't follow the sequence, customers don't see anything until end, testing only at end, assumes natural uncertainty can be eliminated.
- **INCREMENTAL DEVELOPMENT** - based on developing an initial implementation, getting feedback from the users, and evolving the software through multiple versions until system is complete.
- **Incremental vs. Iterative:** each increment adds new features where each iterations improves already existing features, every increment is typically released to users while no iterations are released to users, increment builds everything first then integrate all at once (high risk). Neither of them require planning in upfront.
- **Pipeline:** define services & priorities; define increment 1 requirements; develop and deploy increment 1; user experiments & give feedback; define next increment; repeat until completion.
- **Benefits** are working software is delivered early, lower risk of total project failure, highest priority features receives most testing, easier to manage while
- **problems** are process is less visible, system structure tends to degrade as new features are added, hard to replace and existing system, all the requirements are not defined at the beginning.
- **Use when** working product is required to be released quickly.
- **VERIFICATION & VALIDATION** - **verification:** are we building the product right? **methods to verify:** code reviews & walkthrough, formal inspection, static analysis, checking against de-

sign documents; **validation:** are we building the right product? **methods to validate:** testing with real users, acceptance testing with customer data, user feedback & evaluation, prototype evaluation.

- **THROWAWAY PROTOTYPING** - is built *quickly* with the goal of clarifying ambiguous requirements. it is evaluated by the customer and then discarded. the real system is build by scratch using the knowledge gained.
- **Use when** need to clarify requirements before building the real system.
- **EVOLUTIONARY PROTOTYPING** - the prototype is not discarded. instead, it is continuously refined and evolved based on customer feedback until it becomes the delivered product.
- **Use when** requirements are rapidly changing and unclear.
- **SPIRAL MODEL** - combination of systematic nature of waterfall and iterative nature of prototyping models.
- **Four quadrants:** determine objectives, alternatives, constraints; evaluate alternatives, identify and resolve risks; develop and verify the next-level product; plan the next iteration.
- **Strengths** are risk assessment, prototyping, and customer reviews are built into every loop, combine structure with flexibility.
- **Limitations** are requires expertise in risk assessment, can be difficult to manage, hard to apply to contracts, overkill for small projects.
- **Use when** large and complex systems, long duration projects, safety and security critical system, high risk systems.
- **RAD MODEL** - an incremental software development process model that emphasis an extremely short development cycle. if requirements are well understood and the scope is constrained, RAD creates a fully functional system within 60-90 days. different teams working in parallel on different parts of the system. results are integrated at the end.
- Business modeling, data modeling, process modeling, application generation, testing and turnover.
- **REUSE ORIENTED (component based SE CBSE or integration and configuration)** - rather than developing a software from scratch, the system is assembled from existing reusable components, frameworks, libraries, and commercial-off-the-shelf (COTS) software system. requirements are adapted to fit available components when necessary. This is the mostly used model in modern times.
- **Pipeline:** requirement specification, software discovery and evaluation, requirement refinement, application system configuration, component adaptation and integration.
- **Benefits** are reduces the software that need to develop, reduce cost and risk, faster delivery, higher quality
- **limitations** are requirements compromises, loss of control, security risks, vendor lock-in, integration complexities.

DEVOPS

- DevOps is the culture and set of practices that unifies them around shared responsibility for the full lifecycle.
- **Pipeline:** Plan, Code, Build, Test, Release, Deploy, Operate, Monitor.
- **Continuous integration:** code merged and tested automatically, many times per day. Problems caught early. (Happens in Code, Build, Test phases)
- **Continuous delivery:** software always in a deployable state. Releases are routine, low risk events. (Happens in Release, Deploy)
- **Infrastructure as code:** environments configured through code (containers) deployments are repeatable and auditable. (Happens in Deploy, Operate)

3. AGILE CULTURE

- Embrace change; planning is incremental & continues; working software delivered at each cycle; customer collaboration over negotiation; small teams; best when requirements are unclear. Agile combines both iterative and incremental models, where increment adds features while iterative improves quality. Learn, adapt and deliver.
- **Agile manifesto's four values** are individuals and interactions over process and tools, working software over comprehensive documentation, customer collaboration over contract negotiation, responding to change over following a plan.

Agile manifesto's 12 principles are satisfy the customer, welcome change, deliver frequently, work together daily, trust motivated individuals, face-to-face conversations, working software=progress, sustainable phase, technical excellence, simplicity, self-organizing teams, reflect and adapt.

Use agile when requirements are uncertain, evolving or customer driven; building commercial product; small, co-located, collaborative teams; customer availability for frequent feedbacks; faster delivery is needed; experimental tech stack; high risk.

- **SCRUM** - a lightweight framework that helps teams deliver valuable products through short, repeatable cycles called sprints. it is not a process or a technique, it is a framework within which teams can use various processes and techniques.

Time-boxed sprints: all work happens in 1-4 weeks cycles called sprint. sprint end date never moves. **Inspect and adapt:** at every sprint boundary, the team examines what they built and improves how they work. **Empirical process control:** decisions are based on observations and experimentation.

Five core values: Commitment, Focus, Openness, Respect, Courage

Roles in Scrum: **Product Owner:** owns the backlog. decides what gets build and why. represent customer. **Scrum Master:** coaches the team. removes impediments, protect the sprint from interruptions. **Dev Team:** owns sprint backlog. cross functioning team of 3-9 people.

Artifacts: Product Backlog: ordered list of everything that might be needed in the product. it evolves as the product and market changes. **Sprint Backlog:** set of product backlog items selected for the current sprint plus the plan for delivering them. **Product Increment:** the sum of all the completed backlog items at the end of each sprint. **Five events:** sprint(container for all other events), sprint planning(select backlog item), daily standup/daily scrum(what's done what's next), sprint review(demo to customers), sprint retrospective(improve team process).

Sprint Cycle: Product backlog, sprint planning, sprint backlog, sprint, product increment.

Definition of Done (DoD): the shared understanding of what complete means. a product backlog item is not done until it meets every criterion in DoD. It creates transparency, prevents undone work from accumulating as hidden technical debt, prevents conflict between the team.

Strengths of scrum are delivers working software frequently, continuous feedback reduces wrong products from being built, simple to understand, works well with complex products

limitations are requires cultural change, product owner should be available and empowered, produces less documentations, scrum master is misunderstood as project manager, does not scale well for larger teams.

- **EXTREME PROGRAMMING (XP)** - an engineering methodology focused on engineering excellence and high-quality software delivered in frequent, short releases. philosophy is if something works, do it as much as possible.

Testing works? write tests before code; code review works? review code continuously by pairing; design works? refactor and redesign constantly; simple is good? always do simplest thing as possible; short iterations? deliver frequently.

Five core values: Communication, Simplicity, Feedback, Courage, Respect.

12 core practices: test-driven development, pair programming, refactoring, simple design, continuous integration, coding standards, collective code ownership, sustainable pace, on-site customer, planning game, small releases, metaphor.

Key engineering practices: test-driven development, pair programming, refactoring, continuous integration, collective code ownership, small releases.

Strengths are high code quality through TDD, low defect rates, team flexibility through collective ownership, sustainable pace

best suited when small technically strong teams, customer available for frequent interaction, requirement evolve rapidly, engineering quality is the top priority.

- **KANBAN** - is a method of visualizing work, limiting how much work is in progress, and managing the flow of value from request o delivery.

Three properties: visualize the workflow, limit work-in-progress, manage and improve flow.

Kanban board consists backlog, todo, in progress, in review, done.

WIP Limit: there is a limit to how much items allowed in a workflow stage at any time. when a column in board is full, no new work can enter until one exits. This helps to make a bottleneck visible, so team swarms to fix blocked items. other wise people will start many without actually finishing them.

Six practices: visualize the workflow, limit WIP, manage flow, make policies explicit, feedback loops, improve collaboratively.

- **LEAN** - identify and deliver value to the customer. identify every step in the value stream. eliminate every step that doesn't add value. then make the value adding steps flow without interruptions.

Seven principles: eliminate waste, amplify learning, decide a late as possible, deliver as fast as possible, empower the team, build integrity in, see the whole.

Seven types of waste: partially done work, extra features, re-learning, handoffs, take switching, delays and waiting, defects.

- **CRYSTAL** - A family of methods that vary with team size and project.

Use when small to medium teams wanting a lightweight, people centric approach with flexibility on practices.

- **FUTURE-DRIVEN DEV** - work is planned and tracked by individual features.

Five processes: develop an overall model, build features list, plan by feature, design by feature, build by feature.

Use when larger teams in enterprise settings where feature tracking matters.

- **Scaled Agile Framework (SAFe), Large Scale Scrum (LeSS), The Nexus framework (Nexus), DSDM**

- **AGILE SUMMARY:**

Scrum: Best for: product development, Strength: structure + feedback loops, Challenges: role confusion and culture change, Team size: 3-9, Requirements: unclear or evolving.

XP: Best for: high quality engineering, Strength: TDD, pairing, CI quality, Challenges: require strong team discipline, Team size: 2-10, Requirements: unclear or evolving.

Kanban: Best for: service + maintenance, Strength: flow + WIP limits, Challenges: whole system thinking, Team size: any, continuous delivery with no fixed cadence.

Lean: Best for: waste reduction mindset, Strength: whole system thinking, Challenges: experienced practitioners, Team size: any.

Crystal: Best for: people focused projects, Strength: flexible and lightweight, Challenges: less guidance for new teams, Team size: 1-80.

FDD: Best for: large enterprise, Strength: future level tracking, Challenges: more upfront planning needed, Team size: upto 250.

AGILE VS. PLAN-DRIVEN

- **Primary focus:** in plan driven everything is planed at thee beginning while in agile adapts to change.

- **Requirements document:** plan driven uses SRS and agile uses product backlog.

- **Design timing:** complete design is completed in plan-driven while in agile only the required design is completed per sprint.

- **Customer involvement:** in plan-driven customer is involved primarily at milestones and in agile customer is continuously involved.

HYBRID MODELS

- **UNIFIED PROCESS (UP)** - it combines waterfall's structured phases and milestone reviews (plan-driven culture) with iterative, incremental development within each phase (agile culture).

Inception phase(establish business scope), elaboration phase(requirement analysis), construction phase(development), transition phase(deployment).

RISK MANAGEMENT

- **Technical risk:** can we build this at all? does the technology exists? *How to resolve:* proof-of-concept prototypes.

- **Requirement risk:** do we understand what customer really needs? *How to resolve:* throwaway prototypes evaluated by customer.

- **Schedule risk:** can we deliver in time available? *How to resolve:* realistic planning and milestone tracking.

- **Budget risk:** will the cost exceed funding? *How*

to resolve: early cost estimation with regular review.

- **Integration risk:** will the component work together as a complete system? *How to resolve:* integration prototyping and interface specification.

4. REQUIREMENT ENGINEERING

- Requirement is a condition or capability that must be satisfied. Requirement can take three forms:

User Need: expressed in users own language (i want to do this).

System Rule: what system must provide in order to meet the need (the system shall...).

Documented Form: a written version of either of above.

- **Requirements changes** due to evolving business needs, stakeholder diversity, technology shifts, understanding deepens.

- **Lifecycle of a requirement:** need identified, problem understood, requirement defined, solution proposed, conflicts managed, prioritized.

- **Requirement ambiguity** comes when different people understanding the requirements differently. to remove this make sure to make every word in the requirement means the same thing to every reader.

- **Properties of good requirement:** Clear, Complete, Consistent, Verifiable, Traceable, Feasible.

- **Requirement Development:** feasibility study, elicitation & discovery, analysis & modeling, specification, validation.

- **Feasibility study:** Technical, Economic (budget and ROI), Operational (users), Legal, Schedule (timeline), Resource (people, tools and infrastructure), Cultural (organization culture), Market/Real Estate (location), Financial (funding and financial flows).

- **Requirement elicitation:** gathering requirements from stakeholders.

Elicitation is hard due to stakeholders often don't know what they want, different stakeholders has conflicting ideas, organizational politics.

Fact finding: systematically collect information about organization, stakeholders, and existing system. deliver problem context, system scope, business objectives, existing processes. It focuses on mapping current operations, organizational hierarchy, business goals, and system boundaries. Key activities include identifying stakeholders, interviewing personnel, analyzing legacy systems, and observing current workflows. It establishes project scope early to minimize requirement instability and provides a clear foundational map for guided elicitation. Key deliverables include a problem context statement, defined system scope boundaries, a list of business objectives, and preliminary risk assessments. It acts as the initial entry point to elicitation by mapping current operations and establishing system scope.

Domain analysis: gain deep knowledge on how the organization actually runs to understand its concepts, rules, processes, and constraints. objectives are to understand environment, identify domain concepts, recognize domain specific constraints, discover reusable knowledge from previous projects. this speed up requirement elicitation, help anticipate future needs and changes, reduce miscommunication. It focuses on deepening knowledge of industry rules, legal regulations, operational workflows, and domain terminology. Key activities include analyzing domain terminology, studying business rules, assessing regulatory constraints, and discovering reusable components. It accelerates elicitation, reduces miscommunication between non-technical stakeholders and engineers, and helps anticipate future system needs. Key deliverables include a domain glossary, business rule models, industry constraint specifications, and reusable domain knowledge. It operates alongside fact-finding to build domain literacy, enabling engineers to accurately interpret and validate stakeholder needs.

Four activities of elicitation: discover, classification & organization, prioritization & negotiation, specification.

Discovery techniques: conversational (interviews, workshops, focus groups, brainstorming : qualitative understanding), observational (direct observation, shadowing, ethnography : what people actually do), analytical (document analysis, system archeology, competitor analysis : existing system), synthetic (prototyping, surveys/questionnaires, scenarios, personas : reach-

ing many stakeholders at once, validating).

- **Requirement analysis & modeling:**
Analyzing achieves: detect inconsistent, omissions, validate requirements against scope, establish clear relationship among requirements, provide foundation for design.
Modeling techniques: use case diagrams, activity diagrams, class / er diagrams, data flow diagrams, system context diagrams, state machines.

- **Requirement classification:** All functional, non-functional requirements are required no optional requirements.

Functional requirements: what the system must do.

Non-Functional requirements: how the system should do it (performance, security, reliability, usability, maintainability, interoperability). they should be measurable. (15% faster)

Domain: why the requirement exists (rules, laws and regulations).

Design constraints: technology, standards, or regulatory limits.

Environmental constraints: physical, hardware or network conditions.

- **Requirement prioritization: (MoSCow):**
Must have: essential features. **Should have:** important but not vital. **Could have:** nice to have. **Won't have:** explicitly agreed not to build.

- **Requirement specification:** documenting requirements clearly so designers, developers, and testers can execute them.

Three levels of formality: natural language (user stories), structural language, graphical models (UML diagrams)

User stories format: "As a [role], I want [goal], so that [benefit]"

Acceptance criteria: "Given [initial state, precondition] When [action or event] Then [expected system response]"

System Requirement Specification (SRS): plan drive, written before development, everyone has access, changes have to be reviewed before adding. **Typical structure of SRS:** Introduction (purpose and scope), overall description (system context, assumptions and constraints), user requirements, system requirements, models and diagrams (UML diagrams), appendices (references, version history).

Product Backlog: agile, written incrementally, only dev team and product owner has access, changes can be made anytime.

Problems in requirement documents: mixing design and requirements (writing HOW not WHAT the system do), vague, untestable language (words like fast, user-friendly, robust which means different things to different people), missing requirements (nobody documents something that everyone knows), inconsistency (one section says something and another says opposite)

- **Requirement validation:**

5C validation check: correct, complete, consistent, capable, checkable. same as verification & validation.

Other techniques: requirement reviews, prototyping, test-case generation, acceptance criteria, walkthroughs.

- **Requirement Management:**

- **Change management:** when change is proposed, evaluate the impact, cost and risk. only implement approved changes.

Process: Submit change request, analyze the impact, make the decision, implement if approved, verify and close.

- **Requirement traceability:** every requirement is linked to the stakeholder need that drove it.
Forward traceability: from requirement to design elements, code segments, and test cases.
backward traceability: from requirement to stakeholder needs or objectives.

5. SOFTWARE DESIGN

- **Top down design:** architectural design (handle different features/subsystems), component design (components of each feature/subsystem), interface design (user interface and system interface), data design (databases), algorithmic design.
- **Five fundamental principles:** abstraction, modularity (dividing into subsystems), cohesion (relation between components inside a module), low coupling (changes in one subsystem is unlikely

to force changes on another), information hiding (internal implementation details are hidden).

- **Architecture patterns:**

01 - layered architecture: divides the system into horizontal layers. each layer provides a service to the one above and uses services from the one below.

02 - client server: one (server) processes provided services, another (client) requests them.

03 - model-view-controller (MVC): model (data and business rule) view (what the users see) controller (handles user actions, coordinates model and view).

04 - repository: components do not interact with each other directly. all shared data is held in a central repository. independent read and writes.

05 - pipe & filter: functional transformations process their inputs to produce outputs.

06 - microservices: each feature of a system runs as its own small, independent service. services connected via APIs.

- **Approaches in software design:** procedural (step by step), object oriented (objects and their relationships), component-based (splitting the system), layered/tiered (responsibilities of each layers), service-oriented/API-based (modules communicates), event-driven, functional & data centered (organized data)

6. IMPLEMENTATION

- **Factors to consider when selecting a programming language:** **technical factors:** (domain fit, performance, safety and security, platform and interface, library ecosystems) and **non technical factors:** (team expertise, timescales, maintenance, organisation standards, morale).

- **Programming paradigms:** procedural/structural, object oriented, functional, declarative.

- **Features of a good code:**

Pillars good code: reliability, readability, reuse.

Reliability: The code is fault-free and behaves predictably under unexpected conditions

Readability: Written for the next developer who has to maintain or review it

Reusability: Built from independent components that can be reused across the project or in future applications without rewriting logic from scratch

Design principles and architectures:

High cohesion: Every module, class, or function has one clear purpose and does it well

Low (Loose) Coupling: Interdependence between components is minimized so changing logic in one module does not unexpectedly break another

Abstraction & Information Hiding: Unnecessary details are hidden behind clear public interfaces so external code cannot corrupt internal states

Modularity & Simplicity: The system is divided into manageable, independent pieces without unnecessary complexity or hard-coded values

SOLID principle:

Single Responsibility: A class or function should have only one reason to change

Open / Closed: Open for extension, but closed for modification

Liskov Substitution: Subclasses must be substitutable for parent classes without breaking program execution

Interface Segregation: Prefer multiple specific interfaces over a single large, general-purpose interface

Dependency Inversion: Depend on abstractions (interfaces) rather than concrete implementations

Common ways technical debt accumulates:

Skipping tests: every new feature build on untested code multiplies the debt.

Copy-paste programming: duplicated code means every bug fix and change must be made in multiple places.

Ignoring linter warnings: each ignored warning is known problem left in the code.

Hard coding values: putting specific values directly in code.

- **Fault avoidance:** aim to write code that contains no errors in the first place.
- **Fault tolerance:** accept that some errors will occur and write code that handles them.
- **Exception handling:** lets the program respond to unexpected errors without crashing.
- **Why code reviews matter:** catches bugs early,

spreads knowledge, enforces standards, improves design, builds team culture. (Uses code walkthroughs and code inspections)

7. TESTING

- **Fundamental approaches to testing:**

Black box testing: internal code is not visible. based on specification. find missing features, wrong outputs, specification failures.

White box testing: internal code is visible. based on code structure. find logical errors, dead code, uncovered branches.

- **Test design techniques:**

Equivalence partitioning: divide into groups that behaves the same way and only test one from a group.

Boundary value analysis: try minimum, maximum and just beyond the input range.

Use case based testing: divide test cases from use cases and each test case will tested with real users.

- **Unit testing:** each individual unit (functions, classes, modules) tested independently against specifications. *Conducted by developers. Catches logic errors, off-by-one bugs, null references, boundary violations. white box testing. Features of a good unit test:* fast, isolated, repeatable, automated, self-checking.

Automated Unit testing: done without human interaction. main purpose is to make sure bugs don't reach deployment.

- **Integrated testing:** individual modules are connected and tested together. check for integration and data flow. *Conducted by developers / QA teams. Catches interface bugs, data format mismatches, incorrect API calls, timing issues. both black box & white box testing.*
- **System testing:** complete integrated system is tested against all the requirements. *Conducted by QA teams. Catches missing features, performance failures, security weaknesses, usability problems. black box testing.*
- **Acceptance testing:** customer tests system with real world data to verify specifications. *Conducted by customers / users. Catches requirements misunderstandings, missing business rule, usability gaps. black box testing. Process:* define acceptance criteria, plan tests, derive tests, run tests, negotiate results, accept or reject.

- **Regression testing:** when fixes are done on later stages, this test is carried out to find out if it broke anything else. *Conducted by developers / QA teams. Catches unintended side effects, accidental breakage, new bugs on previously working features.*
- **Release testing:** validate the release candidate requirements, scenarios, performance. *Conducted by separate test team. black box testing.*
- **Development testing:** all the testing covered by dev team (unit, integrated, system).
- **Test Driven Development (TDD):** you never write production code unless you have a failing test first. each small cycle takes just a few minutes. Red (test for small piece of new behavior. it must fail), Green (write minimum code to pass the test), Refactor (cleanup code while keeping all the tests passed).

Benefits of TDD: code coverage, fewer defects, better design, living documentation, regression protection.

- **Test case structure:** Test ID, Description, Expected Output/Input, Pass/Fail result.
- **Goals of testing:** defect discovery, validation (exhaustive testing is impossible.)

OPERATION & MAINTENANCE

- **Corrective Maintenance (20%):** fix bugs found by real users in the live system.
- **Adaptive Maintenance (25%):** adapt the system to new hardware, os updates, changed regulations or APIs.
- **Perfective Maintenance (50%):** improve performance, add new features, improve usability based on customer feedbacks.
- **Preventive Maintenance (5%):** improve reliability, maintainability before failure (refactoring).

AI IN SE

- **AI in Software Engineering:** AI tools (Copilot, ChatGPT) generate boilerplate, suggest completions, explain code, help documentation/tests. Engineering still essential for: understanding require-

ments correctly, designing reliable/scalable architecture, maintainability/security, thorough testing, professional responsibility.

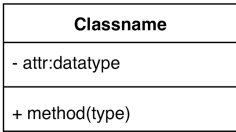
- **AI in Software Processes:** AI assists: Requirements (extract/structure from transcripts, detect conflicts), Design (candidate architectures, flag issues), Coding (Copilot/ChatGPT boilerplate, completion, explain code), Testing (generate test cases, detect anomalies), Operations (AIOps – analyse logs, predict failures, auto-scale). AI cannot: understand context, exercise judgement, take responsibility, resolve ethics – remains engineer's domain.
- **AI in Agile:** Story refinement, effort estimation, dependency detection, risk identification, capacity suggestions.
- **AI in Requirements Engineering:** AI does well: detecting ambiguous language, drafting user stories, suggesting acceptance criteria, summarising interviews, checking format standards. Humans remain essential: prioritising stakeholder needs, negotiating conflicts, ethical/legal judgement, building trust, accountability. Cautions: AI can invent requirements (always trace to a real source); fluent output looks authoritative (stay critical); AI cannot validate its own output (independent human review needed); confidentiality risk; engineer remains responsible.
- **AI in Software Design:** AI helps with: code/class generation, architecture review (identify problems/trade-offs), UML diagram generation from text, design-pattern suggestions, refactoring assistance. What AI cannot do: evaluate business context; avoid over-engineering (engineer must judge simplicity); guarantee reviewed/understood output; avoid propagating bad habits from training data; take professional accountability.
- **AI in Implementation:** Shift: from writing every line to crafting prompts, reviewing/refining AI output; requires MORE deep problem understanding, code reading/evaluation, testing rigor, and architecture judgement – not less. Tools: GitHub Copilot, Cursor, ChatGPT/Claude, Amazon CodeWhisperer, Tabnine. AI does well: boilerplate/scaffolding, explaining unfamiliar code, generating unit tests, refactoring suggestions, documentation, debugging help. AI gets wrong: **hallucination** (invents non-existent functions), security vulnerabilities, wrong logic with correct syntax, outdated patterns/deprecated APIs, licence/copyright issues. Engineer's responsibilities remain: own the outcome, define the problem, validate correctness, make ethical decisions, understand everything submitted (submitting ununderstood code = professional negligence).
- **AI in Software Testing:** AI helps: test case generation (from code/spec, incl. edge cases), code-to-test generation (Copilot/Diffblue), bug prediction (flags risky code areas), anomaly detection (production monitoring), visual regression testing (screenshot diffing). AI gets wrong: tests that always pass (weak/trivial assertions), missing business rules (if code lacks a rule, AI can't test it), wrong assertions (verify expected values!), incorrect mocking, no performance/load tests generated. Engineer's responsibilities: define what "correct" means, verify expected values, identify missing business-level scenarios, review for completeness (mutation testing helps), own the overall test strategy, keep tests updated as code changes.

UML DIAGRAMS

- **Use case diagram:**
system boundary: actors are outside (rectangle)
actors: users of the system (stick man)
use case: ellipse
association: shows interaction between actors and use cases (straight line)
include: must happen every time base use case occur (dotted arrow towards included use case
---<<include>>--->)
extend: may happen every time base use case occur (dotted arrow towards base use case
<--<<extend>>-----)
generalization: when actors or use cases inherit behavior from others (arrow towards parent
←-----).

- **Class diagram:**
Class notation: class name: name of the class,

class attributes: variables taken as parameters, class operations: functions inside.
Visibility: - private, + public, # protected, ~ package

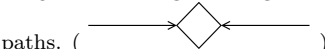


dependency: relationship between classes (----->) (*how to identify:* relies on, calls, requires)
generalization: relationship between sub class and super class (----->) (*how to identify:* is a, is a kind of, types of)
association: shows interaction between classes (straight line) (*how to identify:* borrows, processes, works for, any other verb)
aggregation: whole and part relationship. like a car and its door. (----->) (*how to identify:* has, contains, holds)
composition: strong form of aggregation. whole class solely owns the part class. without whole class, part class cannot exist. like a room inside a building. (----->) (*how to identify:* consists of, made up of)
multiplicity: represent the number of objects participated in a relationship. (exactly *n*: *n*, *n* or more: *n*..*, *n* to *m*: *n*..*m*)

- **Sequence diagram:**
lifeline: represent individual participant in the sequence. (dashed line)
actors: users of the system (stick man)
objects: other objects are represent in a rectangle.
activation bar: shows the period during which an object performs an action. (thin rectangle on the life line of that object ----->)
synchronous message: message that requires a reply before continuing the next action. (----->)
asynchronous message: message that doesn't requires a reply. (----->)
return message: message that return value to the sender. (-----<)
self / reflexive message: object sends a message to itself. (arrow pointing to itself. shape is the same as synchronous message)
alternative: conditional statement.



- **Activity diagram:**
swimlane: activities are divided into groups by responsibility.
item/action: an item is a set of actions and an action is a task to be performed. (rectangle with rounded corners)
control flow: sequence of execution. (----->)
object flow: flow from one activity to another. (----->)
initial and final nodes: start of the activity diagram (●) and end of the activity diagram. (●)
decision node: conditional statement. (----->)
merge node: bring back together different decision



paths. (----->)
fork node: split behavior into parallel activities.



(----->)
join node: bring back together different parallel



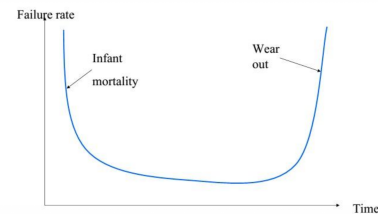
paths. (----->)

SUMMARY

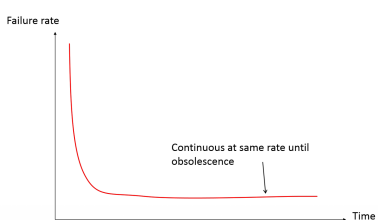
- **stable requirements, contract, regulation :** waterfall, plan driven.
- **risk each cycle :** spiral.
- **deliver pieces, feedback :** incremental.
- **unclear requirements, UI :** prototyping.
- **existing components, COTS :** reuse-oriented.
- **continuous flow, support :** Kanban.
- **engineering practices, TDD, pair :** XP.
- **roles, events, backlog, sprint :** scrum.
- **WHAT service :** functional requirements.
- **HOW WELL, quality :** non-functional requirements.
- **law, business-domain rule :** domain requirements.
- **who can do what :** use case diagram.
- **entities + relationships :** class diagram.
- **manages in order :** sequence diagram.
- **workflow, decision :** activity diagram.
- **software modules :** component diagram.
- **hardware, runtime placement :** deployment diagram.
- **shared central data :** repository architecture.
- **many remote clients, common service :** client-server architecture.
- **multiple UI, separate presentation :** MVC architecture.
- **presentation-business-data layers :** layered architecture.
- **I want maximum adoption-anyone can use it for any purpose :** MIT or Apache 2.0
- **I want it open, but I also want patent protection for users :** Apache 2.0
- **I want all derivative work to remain open source too :** GPL
- **I want companies to use my library but modifications stay open :** LGPL
- **I want to keep it private and own all rights :** Proprietary
- **specification only :** black box testing.
- **branches, paths, code :** white box testing.
- **after code change :** regression testing.
- **customer decision to deploy :** acceptance testing.

IMPORTANT GRAPHS

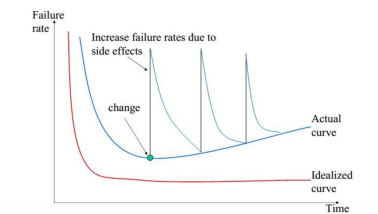
Failure "Bathtub" curve for hardware



Failure curve for software



Failure curve for software: in reality



Cost of Hardware vs. Software

